sequentially passes its input through the layers of the network, and the output of the final layer is what the neural network produces. In PyTorch, each layer is implemented as a *module*, which is a term to denote the basic building block of any object in PyTorch that processes data.

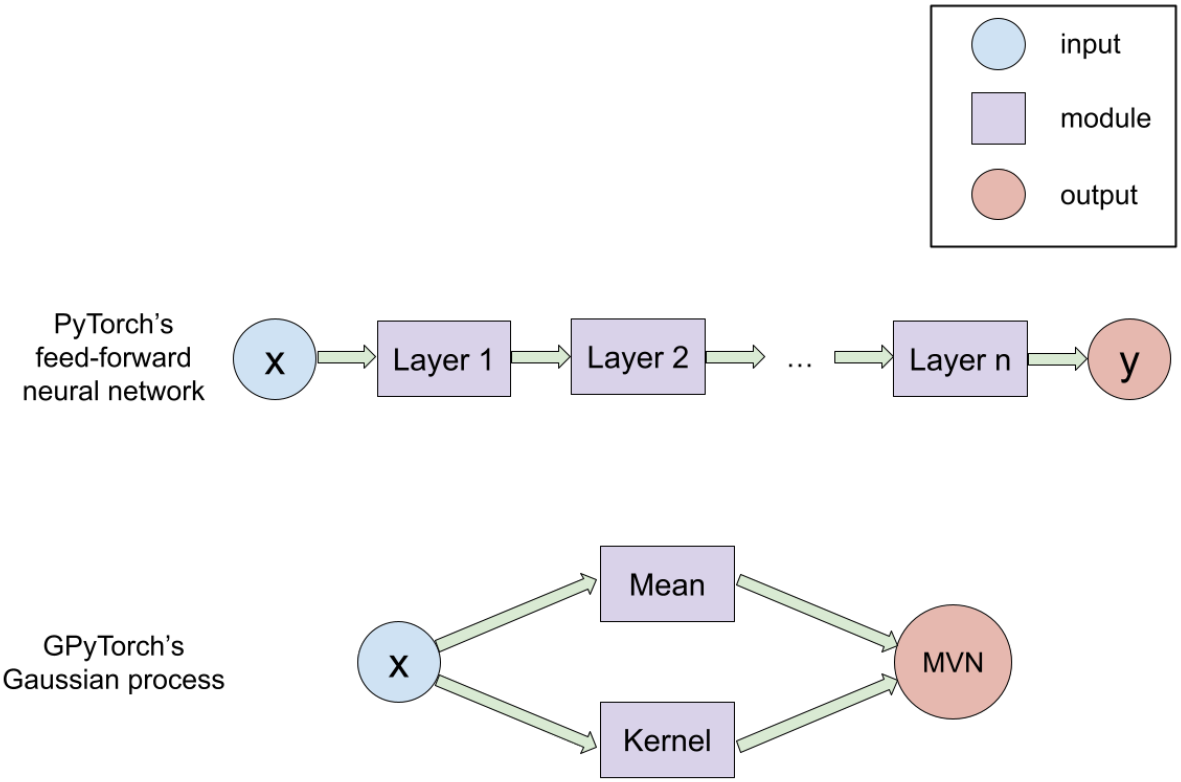


Figure 2.16 How PyTorch and GPyTorch process data in their respective `forward()` methods. The input is processed by different modules to produce the final output, either a number for a feed-forward neural network or a multivarate Gaussian distribution (MVN) for a Gaussian process.

The `forward()` method of a GP in GPyTorch works in a similar way, where the GP's mean and covariance functions are implemented as modules, and the input of the method is passed to these modules. Instead of sequentially passing the result through different modules, we pass the input to the mean and the covariance functions simultaneously. The output of these modules is then combined to create a multivariate Gaussian distribution. This difference between PyTorch and GPyTorch is illustrated in Figure 2.16.

The `forward()` method is implemented in the following code:

```
def forward(self, x):
    mean_x = self.mean_module(x)
    covar_x = self.covar_module(x)
    return gpytorch.distributions.MultivariateNormal(mean_x, covar_x)
```

The logic here is pretty straightforward: as we have a mean function and a covariance function, we simply call them on the input `x` to compute the mean and covariance predictions. Finally, what we need to return is a multivariate Gaussian distribution, implemented by the `gpytorch.distributions.MultivariateNormal` class, with corresponding mean and

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>